Objects Deep Dive

Object Size, Call by Value, Shallow vs Deep Copy

1. How Much Memory Does an Object Take?

Primitive types have a fixed size (`int` = 4 bytes, `double` = 8 bytes). Objects have a size too, and there are **two separate things** to measure.

Thing	Where it lives	Size
Reference variable (`s1`)	Stack	**4 bytes** (8 on some JVMs)
The object itself (`new Student()`)	Heap	Depends on the class (calculated below)

📌 **Why 4 or 8?** Most 64-bit JVMs compress references to 4 bytes by default. A few JVMs use 8. This changes from JVM to JVM, so the course uses **4** as the standard.

What is inside an object?

An object is **not** just its fields. Think of a parcel: the items inside are your fields, but the box also carries a **label** and some **filler**.

> **Object size = Header + Fields + Padding**

Part	What it is	Size
Header	Extra information Java keeps about the object (metadata = "data about data")	**12 bytes**
Fields	The variables you wrote in the class	Add up the field sizes
Padding	Empty filler bytes so the total becomes a multiple of 8	0 to 7 bytes

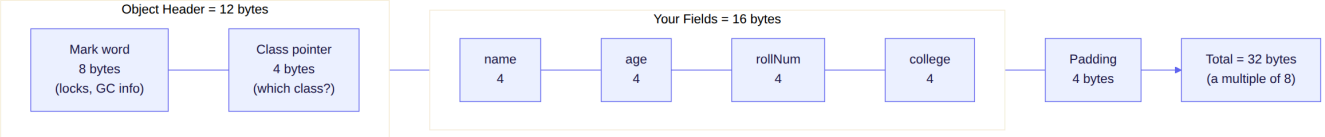
The header has two pieces:

Piece	Job	Size
Mark word	Stores lock, synchronization and garbage-collection information for the object	8 bytes
Class pointer	A reference that tells Java *which class* this object belongs to	4 bytes (8 on some JVMs)

So the header is usually **8 + 4 = 12 bytes**, whatever the class is.

Worked Example: the `Student` class

```
class Student {
    String name;      // reference  -> 4 bytes
    int age;          // int        -> 4 bytes
    int rollNo;       // int        -> 4 bytes
    String college;   // reference  -> 4 bytes
}
```



Step	Calculation	Result
1. Header	8 + 4	12
2. Fields	4 + 4 + 4 + 4	16
3. Header + fields	12 + 16	28
4. Round up to a multiple of 8	next multiple is 32	padding = 32 − 28 = **4**
Total	12 + 16 + 4	**32 bytes**

So `Student s1 = new Student();` uses **4 bytes in the stack** (for `s1`) and **32 bytes in the heap** (for the object).

⚠ **Common Mistake:** Saying a `String` field is "the size of the text". A `String` field is only a **reference**, so it always counts as 4 bytes in the object. The text itself lives elsewhere in the heap.

Why Padding? (the 8-byte rule)

Modern CPUs pick up data from memory in chunks of **8 bytes**. If every object starts and ends on an 8-byte boundary, the CPU can read it cleanly in whole chunks. Padding is just the filler that makes this work.

Second Example: a very small class

```
class Person {
    byte age;    // 1 byte
}
```

Header 12 + field 1 = **13**. The next multiple of 8 is 16, so padding = **3**. **Total = 16 bytes**, even though the class holds only a single byte of real data.

📌 **Important:** In the lecture's handwritten notes, this total is written as "18 bytes". That is an arithmetic slip. 12 + 1 + 3 = **16**, and 18 can't be right because it is not a multiple of 8.

Quick recipe for any class

Header 12 + **field sizes** (`byte` 1, `short` / `char` 2, `int` / `float` 4, `long` / `double` 8, any reference 4), then **round up to a multiple of 8**. (With 8-byte class pointers the header is 16 and the numbers shift.)

---

2. Call by Value (Primitives)

Look at this program. What do you expect it to print?

```
public class Demo {
    public static void main(String[] args) {
        int x = 4;
        int y = 5;
        System.out.println(x + " , " + y);   // 4 , 5

        addTen(x, y);

        System.out.println(x + " , " + y);   // 4 , 5  (not 14 , 15!)
    }
```

```
        static void addTen(int x, int y) {
            x = x + 10;
            y = y + 10;
        }
    }
```

Output:

```
4 , 5
4 , 5
```

Why didn't the values change?

Step	What happens
1	`main` creates its own boxes `x = 4` and `y = 5`
2	`addTen(x, y)` is called. Java creates **two brand new boxes** named `x` and `y` for `addTen` and **copies** 4 and 5 into them
3	`addTen` adds 10 to **its own** boxes (they become 14 and 15)
4	`addTen` ends. Its boxes are **erased** from memory
5	Back in `main`, the original boxes still hold 4 and 5

The two `x` variables only share a **name**. They are completely different boxes in memory.

> **Definition: Call by value** means a method receives a **copy** of the value you pass. Changing the copy never changes the original.

💡 **Easy Hinglish Explanation**

> "Call by value matlab method ko value ki photocopy milti hai, original nahi. Method photocopy pe kuch bhi likhe, original paper waisa ka waisa rehta hai."

---

3. Call by Reference (and Why Java Doesn't Have It)

If `addTen`'s `x` and `y` could point to the **same boxes** as `main`'s `x` and `y` (instead of new copies), adding 10 inside `addTen` would change the originals and the program would print `14 , 15`. That idea is **call by reference**: the method gets the *original variable itself*. Languages like **C++** allow it.

> ⭐ **Java has NO call by reference. Java is only call by value. Always.**

---

4. Passing Objects to Methods

Now the confusing part. Here the values **do** change:

```
    public class Demo2 {
        public static void main(String[] args) {
            Random r1 = new Random(4, 5);

            System.out.println(r1.x + " , " + r1.y);   // 4 , 5
            addTen(r1);
```

```
            System.out.println(r1.x + " , " + r1.y);   // 14 , 15
        }

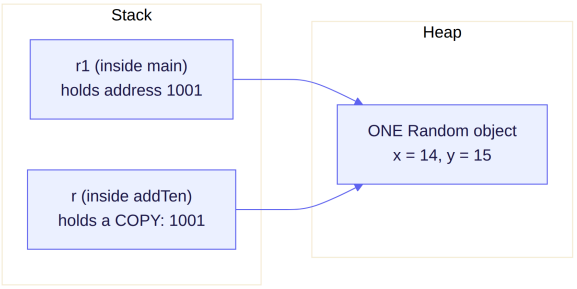
        static void addTen(Random r) {
            r.x = r.x + 10;
            r.y = r.y + 10;
        }
    }

    class Random {
        int x;
        int y;
        Random(int x, int y) {
            this.x = x;
            this.y = y;
        }
    }
```

Output:

```
4 , 5
14 , 15
```

It looks like call by reference, but it is still **call by value**. Here is why.



Step	What happens
1	`new Random(4, 5)` creates **one object** in the heap. `r1` (in the stack) stores its address, say `1001`
2	`addTen(r1)` is called. Java creates a **new reference variable** `r` and **copies** the value of `r1` into it. The value of `r1` is an **address**, so `r` now also holds `1001`
3	`r.x = r.x + 10` follows the address to the heap object and changes it. There is only one object, so this changes the real data
4	`addTen` ends. `r` is erased. **The object stays**, because `r1` still points to it
5	`r1.x` and `r1.y` now show 14 and 15

📌 **The key idea:** Java copied the **reference** (the address), not the **object**. Two arrows now point to one object. The *variable* was passed by value. The *object* was shared.

💡 **Easy Hinglish Explanation**

> "Object pass karte waqt Java object ki copy nahi banata. Wo sirf address ki copy banata hai. Dono variables (r1 aur r) ek hi object ko point karte hain, isliye method ke andar kiya gaya change bahar bhi dikhta hai."

Proof that it is not call by reference

If Java were truly call by reference, then pointing `r` at a **new** object inside the method would also move `r1`. Let's test:

```
static void reset(Random r) {
    r = new Random(0, 0);    // r now points to a NEW object
}

// in main:
Random r1 = new Random(4, 5);
reset(r1);
System.out.println(r1.x + " , " + r1.y);   // 4 , 5
```

`r1` is **unchanged**. Only the method's own copy `r` was redirected to the new object. The caller's `r1` still points to the old one.

Primitives vs Objects: one rule for both

	Passing `int x`	Passing `Random r`
What gets copied	The number	The address
Can the method change the caller's *variable*?	No	No
Can the method change the *object's fields*?	n/a	**Yes** (the address leads to the same object)
Java passes it by	**Value**	**Value**

⚠ **Interview trap:** Many freshers say *"Java is call by value for primitives and call by reference for objects."* This is **wrong**. The correct answer is: **Java is always call by value. For objects, the value being copied is the reference (address).**

Objects can also be a return type (`static Random makeRandom()`) and a **constructor parameter** (`Random(Random r)` , used for copying below).

---

5. Shallow Copy vs Deep Copy

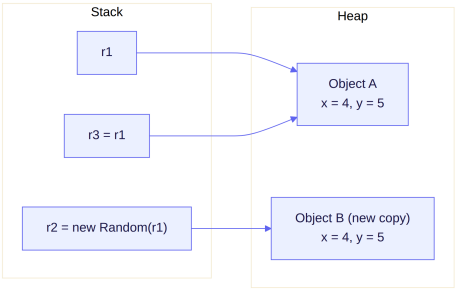
Suppose you have `r1` with `x = 4, y = 5`, and you want "another one". There are two very different ways to do it. For this we add a **copy constructor** to the class, a constructor that takes an object of its own class:

```
class Random {
    int x;
    int y;

    Random(int x, int y) {       // normal constructor
        this.x = x;
        this.y = y;
    }

    Random(Random r) {           // copy constructor
        this.x = r.x;
        this.y = r.y;
    }
}
```

```
Random r1 = new Random(4, 5);
Random r2 = new Random(r1);   // Way 1: build a NEW object and copy the values
Random r3 = r1;               // Way 2: copy only the address
```



	`r2 = new Random(r1)`	`r3 = r1`
Name	**Deep copy**	**Shallow copy**
Uses `new` ?	Yes, a new object is created	No, no new object
Objects in heap	**Two** (A and B)	**One** (A)
What `r2` / `r3` holds	Address of its own object	The **same address** as `r1`
Change `r3.x = 14` / `r2.x = 14`	`r2` changes alone. `r1.x` stays 4	Changes `r1.x` too (same object)

Test it: after `r2.x = 14`, `r1.x` is still **4**. After `r3.x = 14`, `r1.x` becomes **14**.

> **Rule to remember:** No `new` = no new object = just another name (arrow) for the same object. `new` = a separate object.

⭐ **Exam Tip:** `r3 = r1` creates a new *reference variable*, but **not** a new *object*. Count the `new` keywords to count the objects.

⚠ **Terminology note:** Other books say a *shallow copy* is a new object whose inner objects are still shared, and a *deep copy* also duplicates those inner objects. The lecture uses the simpler version: **shallow = copy the arrow, deep = copy into a new object**.

💡 **Easy Hinglish Explanation**

> "Shallow copy mein sirf address copy hota hai, isliye dono variables ek hi object ko point karte hain. Deep copy mein naya object banta hai aur values copy hoti hain, isliye dono bilkul alag ho jaate hain."

🌍 Real-Life Examples

Concept	Everyday picture
Call by value (primitive)	You photocopy a form and hand over the copy. Scribbling on it does not damage your original.
Passing an object (copy of a reference)	You share the **link** to a Google Doc. Your friend gets a copy of the link, but both links open the **same** document. When they edit, you see it.
Shallow copy	Two people with the same house address. Painting the house changes it for both.
Deep copy	Building an identical second house. Painting one does not touch the other.
Copy constructor	"Duplicate" in a design app, or "Save As" in a document: same content, separate file.

A shopping app's "reorder" button should build a **deep copy** of the old order, or editing the new cart would silently change your order history.

⭐ One-Page Revision

- **Reference** = 4 bytes (stack). **Object** = Header (12) + Fields + Padding, rounded to a multiple of 8 (heap). `Student` = **32 bytes**, `Person` (one `byte`) = **16 bytes**.
- Java is **only call by value**. Primitive: the value is copied, original unchanged. Object: the **address** is copied, so both variables reach the **same object**.
- Reassigning a parameter (`r = new ...`) never changes the caller's variable.
- `r3 = r1` → shallow copy (one object, two arrows). `r2 = new Random(r1)` → deep copy (two independent objects).

📝 Practice Questions

Concept questions

1. What are the three parts of an object's size in memory?
2. Why does Java add padding to objects?
3. Is Java call by value or call by reference? Explain, including what is copied when an object is passed.
4. What is the difference between `r3 = r1` and `r2 = new Random(r1)` ?

Programming questions

1. **Predict the output.** Then explain why:

```
static void swap(int a, int b) { int t = a; a = b; b = t; }
// main:
int p = 10, q = 20;
swap(p, q);
System.out.println(p + " " + q);
```

2. **Predict the output.** Does the method change `r1` ?

```
static void change(Random r) { r.x = 99; r = new Random(1, 1); r.y = 77; }
// main:
Random r1 = new Random(4, 5);
change(r1);
System.out.println(r1.x + " " + r1.y);
```

3. Write a class `Point` with fields `x` and `y` , a normal constructor, and a **copy constructor**. Create `p1` , make `p2` as a deep copy and `p3` as a shallow copy. Change `p2.x` and then `p3.x` , and print `p1.x` after each change.
4. Calculate the size in bytes (using Header 12 + Fields + Padding) of:

```
class Book { int pages; double price; String title; }
```

5. Write a method `static void doubleAll(int[] arr)` that doubles every element of the array. Call it from `main` and print the array afterwards. Does the original array change? Why? (Arrays are objects.)
6. Write a method `static Point makeCopy(Point p)` that **returns** a deep copy of `p` . Prove with a test that changing the copy does not change the original.
